```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split

# Unified table: text | label (0/1) | source
x_train, x_test, y_train, y_test = train_test_split(
    df["text"], df["label"],
    test_size=0.2,
    random_state=42,
    stratify=df["label"],
)

baseline = Pipeline(steps=[
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=100_000)),
    ("clf", LogisticRegression(max_iter=2000, class_weight="balanced")),
])

baseline.fit(x_train, y_train)
y_pred = baseline.predict(x_test)
```

```python
def get_model_candidates(random_state, max_features=100_000,
                         char_features=65_536, word_min_df=5,
                         char_max_chars=512, use_char_features=True):
    return {
        "logistic_regression": lambda: Pipeline(steps=[
            ("tfidf", build_vectorizer(max_features, char_features,
                                       word_min_df, char_max_chars,
                                       use_char_features)),
            ("clf", LogisticRegression(max_iter=2000,
                                       class_weight="balanced",
                                       random_state=random_state)),
        ]),
        "linear_svc": lambda: Pipeline(steps=[...]),
        "calibrated_linear_svc": lambda: Pipeline(steps=[...]),
        "multinomial_nb": lambda: Pipeline(steps=[...]),
        "sgd_classifier": lambda: Pipeline(steps=[...]),
        "passive_aggressive": lambda: Pipeline(steps=[...]),
    }
```